

ATLAS: Autonomous Telemetry, Logging, Analysis, & Surveillance

Central Server Design Proposal

Proposed by Jishnu Prasad
jishnuprasad.241ee121@nitk.edu.in
9844778936

May 2026

Contents

1	Executive Summary	5
2	System Architecture Overview	6
2.1	High-Level Data Flow	6
2.2	Server Engine Architecture	6
3	Physical Topology	6
4	Authentication & Authorization	8
4.1	Roles	8
4.2	RBAC Enforcement Perimeter	8
4.3	Authentication Token Model	8
4.4	Password Recovery	9
5	Agent Registry Engine	9
5.1	Agent Registration	9
5.2	Agent Lifecycle States	9
5.3	Agent Tagging	9
6	Namespace Manager Engine	10
6.1	Namespace Definition	10
6.2	Namespace Versioning	10
6.3	Namespace Groups	10
7	Execution Coordinator Engine	11
7.1	Execution Lifecycle	11
7.2	Execution Record	11
7.3	Fleet Execution Targeting	11
8	Scheduler Engine	12
8.1	Schedule Types	12
8.2	Schedule Configuration	12
8.3	Conflict and Missed Schedule Policies	12
8.4	Maintenance Windows	12
9	Metrics Aggregation Engine	13
9.1	Aggregation Pipeline	13
9.2	Fleet Metrics Retention	13
10	Log Aggregation Engine	13
10.1	Log Capabilities	14
11	Immutable Audit Engine	14
11.1	Audited Event Categories	14
11.2	Audit Log Schema	14
12	Storage Engine	15
12.1	PostgreSQL Schema Layout	15
12.2	Database Operational Requirements	15
13	NATS Gateway Engine	15
13.1	Transport Architecture	15

13.2 Subject Hierarchy	15
13.3 NATS ACL Enforcement	16
14 Queue Manager Engine	16
14.1 Outbound Command Queue	16
15 Encryption Engine	17
15.1 Key Rotation	17
16 REST API Engine	17
17 WebSocket Gateway Engine	18
17.1 Available Channels	19
17.2 Back-pressure and Flood Protection	19
18 Health Monitor Engine	19
18.1 Server Health States	19
18.2 Agent Fleet Health Summary	19
19 TUI Design	20
19.1 View Inventory	20
19.2 TUI Requirements	20
20 CLI Reference	20
20.1 Installation & Service	20
20.2 Initialization	20
20.3 Authentication	20
20.4 Agent Management	21
20.5 Namespace Management	21
20.6 Group Management	21
20.7 User Management	21
20.8 Scheduler	21
20.9 Database Operations	22
20.10 Queue, Encryption, API, WebSocket	22
20.11 Connectivity & TUI	22
21 Security Architecture	22
21.1 Defense-in-Depth Layers	23
21.2 Brute Force and Session Protections	23
22 Edge Cases & Failure Handling	23
22.1 Agent Connectivity	23
22.2 Execution Coordination	23
22.3 Database and Storage	24
22.4 WebSocket and API	24
23 Middleware Stack	24
24 Deployment Resource Budget	24
25 Non-Functional Requirements	25
26 Implementation Roadmap	26

1 Executive Summary

ATLAS Server is the **central management plane** of the ATLAS platform. It is the authoritative hub through which all Beacon agents — installed on managed Linux machines — are registered, supervised, commanded, and audited. The server is a production-grade, distributed control system written in Rust, designed to manage fleets from a single agent up to 1000+ nodes without architectural changes.

Where the Beacon agent is the *operations platform on each node*, ATLAS Server is the *fleet brain*: the system that aggregates telemetry, enforces organisation-wide RBAC, manages namespace and group definitions, operates the global scheduler, maintains immutable fleet-wide audit trails, and provides a unified REST API and WebSocket gateway for human operators and external tooling.

Architecture Principle. ATLAS Server and Beacon Agent are strict peers connected exclusively through NATS JetStream. The server never issues raw shell commands. All execution flows through versioned, approved namespaces defined on the server and dispatched to agents as typed operation messages. The agent makes no authorisation decisions — those live entirely on the server.

The complete platform consists of two separate Rust binaries:

Binary	Responsibility
beacon-agent	Installed on every managed Linux node. Executes approved operations locally, collects telemetry, maintains SQLite state, buffers offline.
atlas-server	Runs on the central control plane. Manages authentication, RBAC, namespaces, scheduling, metrics aggregation, audit, REST/WebSocket API, NATS gateway.

2 System Architecture Overview

2.1 High-Level Data Flow

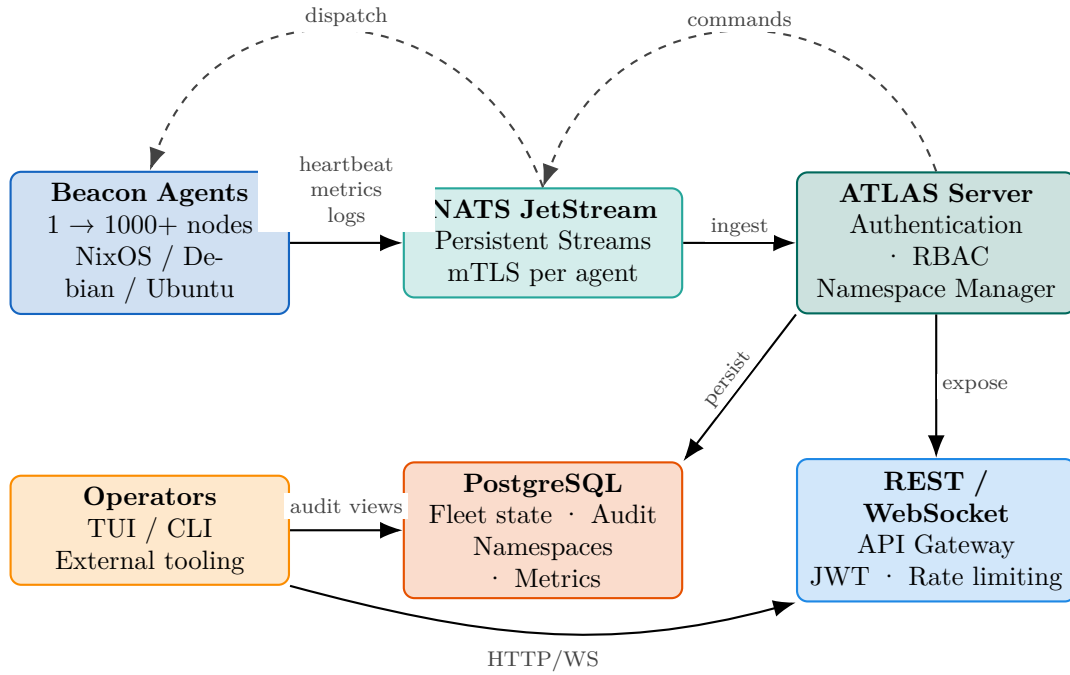


Figure 1: ATLAS Server High-Level Data Flow

2.2 Server Engine Architecture

ATLAS Server is composed of independent, purpose-scoped internal engines — mirroring the engine model of the Beacon agent:

- Authentication Engine
- Authorization Engine (RBAC)
- Agent Registry Engine
- Namespace Manager Engine
- Group Manager Engine
- Execution Coordinator Engine
- Scheduler Engine
- Metrics Aggregation Engine
- Log Aggregation Engine
- Audit Engine
- Queue Manager Engine
- NATS Gateway Engine
- REST API Engine
- WebSocket Gateway Engine
- Health Monitor Engine
- Encryption Engine
- Backup Engine
- Plugin Registry Engine

3 Physical Topology

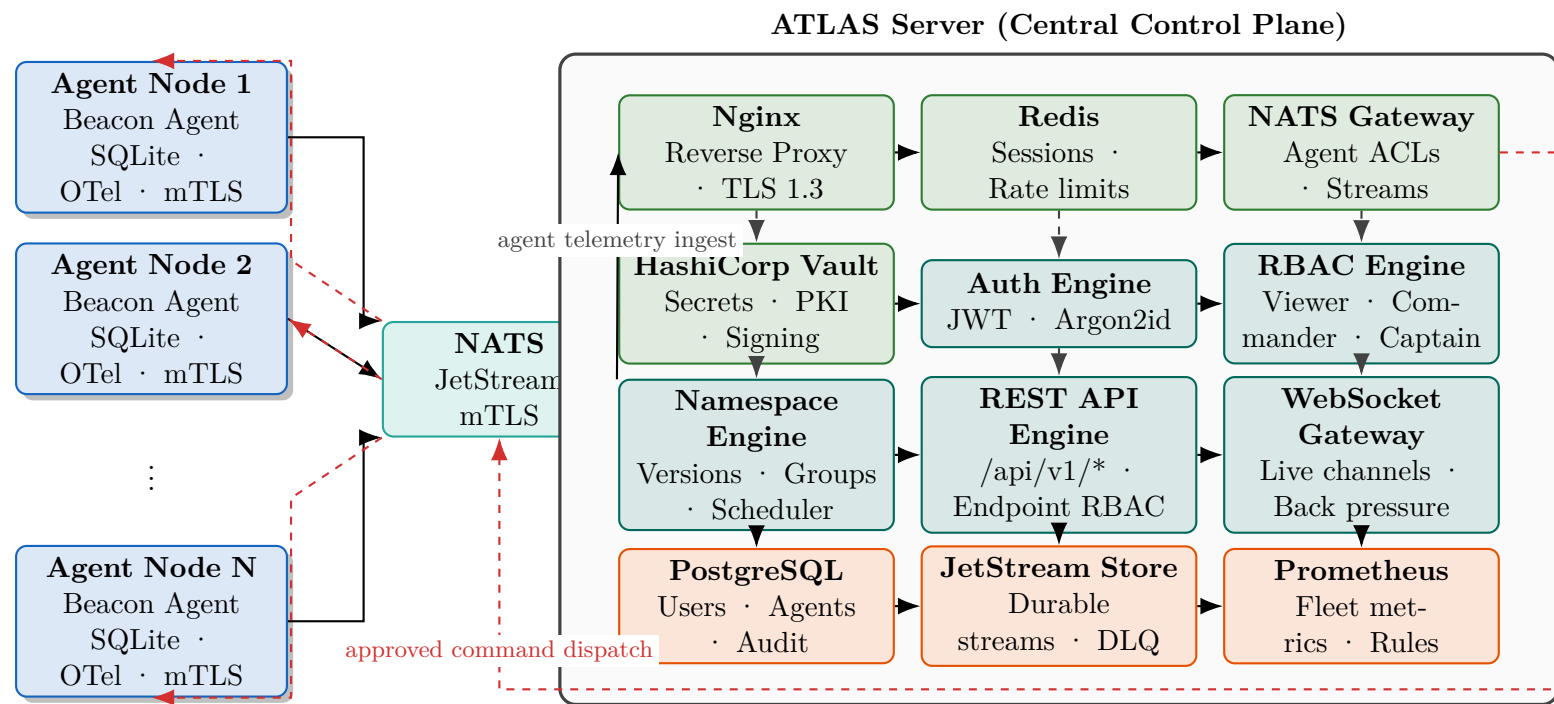


Figure 2: Physical Topology: Beacon Agents → NATS JetStream → ATLAS Server

4 Authentication & Authorization

4.1 Roles

ATLAS Server enforces a three-tier RBAC model organisation-wide. Agent-level namespace RBAC is an additional layer on top.

Role	Scope	Capabilities
Viewer	Read-only	View metrics, logs, namespaces, groups, agents, executions, audit history. Export logs and metrics. No write or execute access whatsoever.
Commander	Execution	Everything Viewer can do. Additionally: execute namespaces and groups against agents and tags, pause/resume/stop running executions. Cannot create, modify, or delete any resource. Cannot manage users or configuration.
Captain	Full admin	Complete control. User CRUD, namespace and group CRUD, scheduling, password management, database operations, backup management, configuration, encryption, metrics config, agent management, audit management.

Table 1: RBAC Role Definitions

4.2 RBAC Enforcement Perimeter

RBAC is validated **server-side at every boundary**. UI-only restrictions are never accepted as sufficient.

- REST API (every endpoint)
- WebSocket subscriptions
- TUI and CLI sessions
- NATS subject publishing
- Scheduler job creation
- Execution Coordinator
- Database write operations
- Configuration mutations
- Backup and restore
- Encryption key operations

Critical Security Rule. Agents never make authorisation decisions. When a command arrives at a Beacon agent via NATS, it has already been validated by the ATLAS Server RBAC engine, signed by the server's Vault Transit key, and logged in the immutable audit trail. The agent verifies the signature and executes; it does not re-evaluate permissions.

4.3 Authentication Token Model

Mechanism	Details
Password hashing	Argon2id; plaintext passwords never stored or logged
Access token	Short-lived JWT signed by Vault Transit key
Refresh token	Long-lived; stored server-side; revocable per session
Session expiration	Configurable; enforced; all login/logout events audited
Agent identity	Per-agent mTLS certificates issued by Vault PKI

4.4 Password Recovery

- **Primary:** Recovery Key generated at server initialisation (format: AB2F-D921-882C-771F). Captain must save it securely before setup completes.
- **Optional secondary:** Security question, configurable post-init; used only as a secondary factor, never as sole recovery mechanism.

All password resets are recorded in the immutable audit log with full operator context.

5 Agent Registry Engine

5.1 Agent Registration

On first startup, a Beacon agent publishes a registration message to NATS. The server processes it and records the agent permanently:

Listing 1: Agent registration payload

```
{
  "agent_id":      "sha256:a3f1c9...",
  "hostname":      "node-prod-07",
  "os":            "NixOS 24.05",
  "architecture": "aarch64",
  "version":       "1.4.2",
  "tags":          ["production", "web", "edge"],
  "last_seen":     "2026-05-31T14:22:00Z",
  "status":        "ONLINE"
}
```

5.2 Agent Lifecycle States

State	Description
ONLINE	Heartbeat received within threshold window
DEGRADED	Heartbeat late or partial telemetry only
OFFLINE_BUFFERING	Agent reports local buffering mode
UNREACHABLE	No heartbeat for configurable timeout
DISABLED	Manually disabled by Captain
DEREGISTERED	Removed from fleet; history retained

5.3 Agent Tagging

Agents carry user-defined tags. The server uses tags as targeting dimensions for namespace execution, group runs, and scheduled jobs:

Listing 2: Tag-based fleet execution commands

```
atlas namespace run update_system --tag production
atlas namespace run audit_logs    --tag database
atlas group      run provision     --tag staging
atlas namespace run reboot_check   --tag raspberrypi
```

Built-in tag vocabulary: production, staging, testing, database, web, edge, raspberrypi. Custom tags are supported without restriction.

6 Namespace Manager Engine

6.1 Namespace Definition

A namespace is a reusable, immutable, versioned workflow definition stored on the server. Agents do not store namespace definitions — they receive typed operation payloads dispatched by the server at execution time.

Listing 3: Server-side namespace definition

```
{
  "namespace":      "update_system",
  "version":        "v3",
  "required_role":  "commander",
  "lock_mode":      "deny",
  "timeout_sec":    600,
  "operations": [
    { "type": "update_packages",    "timeout_sec": 120 },
    { "type": "upgrade_packages",   "timeout_sec": 300 },
    { "type": "autoremove_packages","timeout_sec": 60  }
  ],
  "rollback_policy": "reverse_completed",
  "audit_policy":    "full"
}
```

6.2 Namespace Versioning

Every published namespace version is **immutable**. The server maintains the full version history:

```
update_system_v1  # Original: update + upgrade
update_system_v2  # Added: autoremove step
update_system_v3  # Added: pre-check disk space operation
```

Execution requests reference an explicit version. The server pins that version for the entire execution lifetime. Running executions never switch versions mid-flight. New versions take effect only on subsequent invocations.

6.3 Namespace Groups

The server supports ordered composition of namespaces into groups:

Listing 4: Namespace group definition

```
group: provision_server
steps:
  - namespace: install_packages    version: v2  enabled: true
  - namespace: configure_firewall  version: v1  enabled: true
  - namespace: create_users        version: v3  enabled: true
  - namespace: start_services      version: v1  enabled: true
required_role: captain
execution_mode: sequential
```

Groups support reordering, per-step enable/disable, sequential and parallel execution modes, and group-level execution history.

7 Execution Coordinator Engine

7.1 Execution Lifecycle

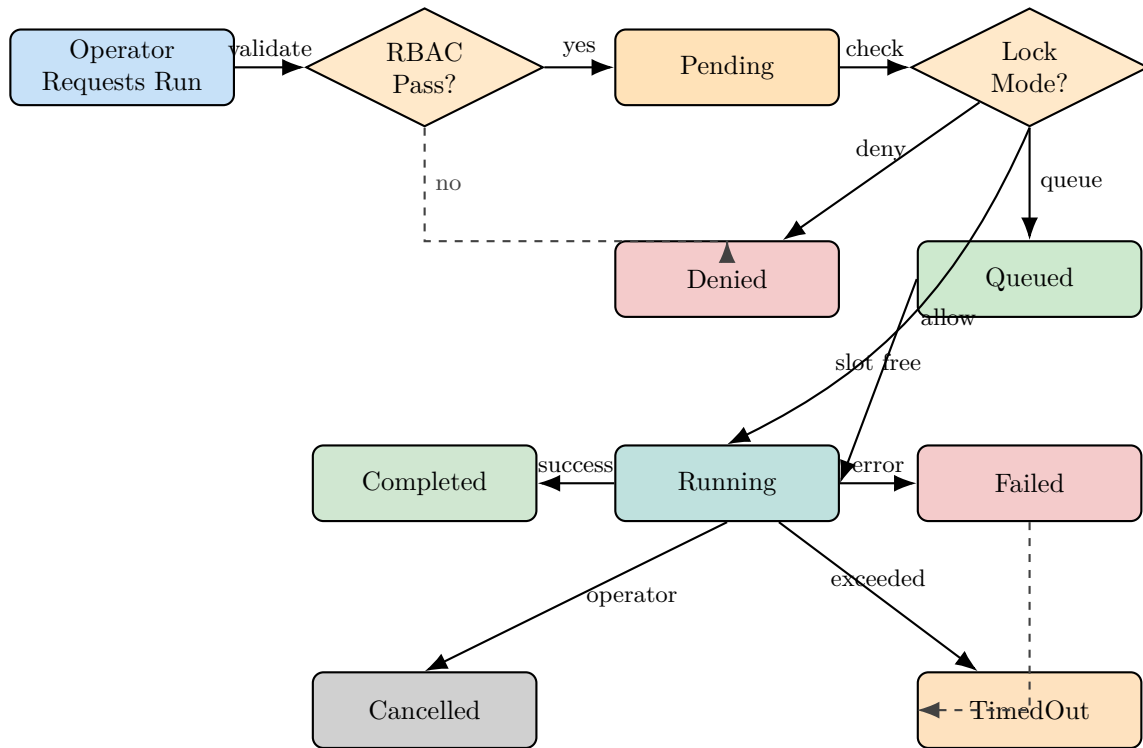


Figure 3: Server-side Execution State Machine

7.2 Execution Record

Every execution is assigned a permanent server-side record:

Field	Type	Description
execution_id	UUID	Globally unique execution identifier
namespace	TEXT	Namespace name
namespace_version	TEXT	Pinned version (v3)
target_agents	TEXT[]	Array of targeted agent IDs
target_tags	TEXT[]	Tags used for targeting (if any)
initiator	TEXT	Authenticated username or scheduler
initiated_at	TIMESTAMPTZ	UTC start time
status	TEXT	Current lifecycle state
completed_at	TIMESTAMPTZ	UTC completion time (nullable)
result_summary	JSONB	Per-agent outcomes

7.3 Fleet Execution Targeting

The Execution Coordinator resolves targets at dispatch time and fans out operation payloads to each qualifying agent via their dedicated NATS subjects:

Listing 5: NATS dispatch subjects

```
# Single agent
beacon.agent.agent001.commands

# Tag-based fan-out (resolved to individual subjects)
beacon.group.production.commands

# Broadcast all
beacon.all.commands
```

8 Scheduler Engine

8.1 Schedule Types

- Every Minute
- Every Hour
- Every Day
- Every Week
- Every Month
- On Agent Boot
- Full Cron Expression

8.2 Schedule Configuration

Listing 6: Scheduled job definition

```
schedule:
  id:          sched-update-prod-001
  namespace:   update_system
  version:     v3
  target_tags: [production]
  cron:        "0 2 * * 0"          # Sundays at 02:00 UTC
  required_role: captain
  missed_policy: ExecuteImmediately
  conflict_policy: Skip
  maintenance_aware: true
```

8.3 Conflict and Missed Schedule Policies

Policy Type	Value	Behaviour
Conflict	Skip	Do not start if a prior execution is still running
	Queue	Enqueue behind current execution
	Replace	Cancel queued; start new
	Parallel	Allow concurrent run
Missed	ExecuteImmediately	Run as soon as the scheduler recovers
	Ignore	Silently skip the missed window
	Queue	Enqueue the missed run

8.4 Maintenance Windows

Maintenance windows suppress dangerous executions on tagged agent groups:

```
atlas maintenance enable --tag production --duration 4h
atlas maintenance disable --tag production
atlas maintenance list
```

Scheduled jobs with `maintenance_aware: true` are automatically held and replayed after window expiry.

9 Metrics Aggregation Engine

9.1 Aggregation Pipeline

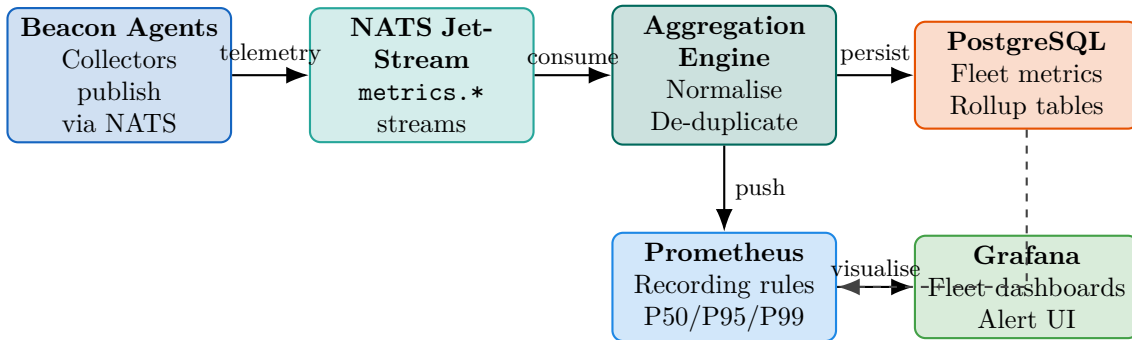


Figure 4: Fleet Metrics Aggregation Pipeline

9.2 Fleet Metrics Retention

Resolution	Interval	Retention
Raw (per agent)	1 second	24 hours
Fleet rollup	1 minute	30 days
Fleet rollup	1 hour	365 days

Automatic pruning runs nightly. Rollups are computed as PostgreSQL recording jobs. Per-agent raw data is compressed with Zstandard before long-term storage.

10 Log Aggregation Engine

The server aggregates structured log entries from all Beacon agents. Agents publish logs to their dedicated NATS stream; the server consumes, normalises, indexes, and stores them in PostgreSQL for full-text search and audit reference.

Listing 7: Aggregated log entry stored on server

```
{
  "server_id": "atlas-server-01",
  "agent_id": "sha256:a3f1c9...",
  "hostname": "node-prod-07",
  "timestamp": "2026-05-31T14:22:01.123Z",
  "execution_id": "exec-00421",
  "namespace": "update_system",
  "severity": "Info",
}
```

```

"source":      "execution_engine",
"message":     "Package upgrade completed"
}

```

10.1 Log Capabilities

```

atlas logs                                # Latest fleet logs
atlas logs follow                        # Live stream (WebSocket)
atlas logs --agent node-prod-07         # Filter by agent
atlas logs --namespace update_system
atlas logs --severity error
atlas logs search "disk full"
atlas logs export --format json --from 2026-05-01
atlas logs clear --agent node-prod-07 --severity debug

```

11 Immutable Audit Engine

Audit records on the server are immutable. Not even the Captain role can delete or modify audit entries. The audit log is the authoritative record of every action taken across the entire fleet. It is stored in an append-only PostgreSQL table with row-level security preventing DELETE and UPDATE at the database level.

11.1 Audited Event Categories

- User login / logout
- Failed authentication attempts
- RBAC violations (denied actions)
- Namespace CRUD operations
- Group CRUD operations
- Execution start / stop / pause
- Scheduler job changes
- Agent registration / deregistration
- Password changes / resets
- User CRUD operations
- Configuration changes
- Encryption key rotations
- Backup operations
- API key issuance / revocation

11.2 Audit Log Schema

Column	Type	Description
id	UUID	Globally unique event identifier
ts	TIMESTAMPTZ	Wall-clock time (UTC)
operator	TEXT	Authenticated username
role	TEXT	Role at time of action
action_type	TEXT	Category from enumeration above
target_type	TEXT	Resource type (agent, namespace, user...)
target_id	TEXT	Resource identifier
payload_hash	TEXT	SHA-256 of full action payload
source_ip	INET	Client IP address
outcome	TEXT	SUCCESS / DENIED / FAILED
detail	JSONB	Structured supplementary data

12 Storage Engine

12.1 PostgreSQL Schema Layout

The server uses PostgreSQL as its primary datastore. Schemas are logically separated by concern:

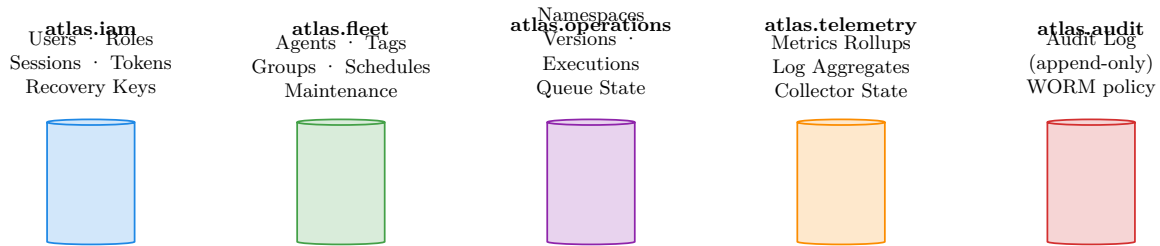


Figure 5: PostgreSQL Schema Layout (five logical schemas, one database)

12.2 Database Operational Requirements

- WAL archiving enabled with point-in-time recovery
- Row-level security on `atlas.audit` prevents `DELETE/UPDATE`
- Automated daily logical backups with integrity verification
- Connection pooling via PgBouncer (transaction mode)
- `VACUUM` and `ANALYZE` scheduled during low-traffic windows
- Read replica available for metrics/log query workloads

13 NATS Gateway Engine

13.1 Transport Architecture

All communication between ATLAS Server and Beacon agents flows exclusively through NATS JetStream. The server never connects directly to agent TCP ports. This decoupling provides natural offline buffering: when an agent is unreachable, messages accumulate in the JetStream durable stream and are delivered when the agent reconnects.

Why NATS JetStream? For fleets of 1–1000+ agents, NATS JetStream provides durable, ordered, at-least-once delivery with dead-letter queues, consumer groups, stream retention policies, and replay — all in a fraction of the resource footprint of Kafka. JetStream’s simplicity makes it operationally sustainable for infrastructure teams of any size.

13.2 Subject Hierarchy

Listing 8: NATS subject naming conventions

```
# Agent → Server (published by agent)
beacon.agent.<agent_id>.heartbeat
beacon.agent.<agent_id>.metrics
beacon.agent.<agent_id>.logs
beacon.agent.<agent_id>.audit
beacon.agent.<agent_id>.execution.result

# Server → Agent (subscribed by agent)
beacon.agent.<agent_id>.commands
```

```

beacon.agent.<agent_id>.config.update

# Server → Tag group (fan-out)
beacon.group.<tag>.commands

# Server → All agents
beacon.all.commands
beacon.all.config.update

```

13.3 NATS ACL Enforcement

The NATS Gateway Engine enforces per-agent ACLs issued by the server's Vault PKI. Each agent may only publish to its own subjects and subscribe to its own command channel. Cross-agent subject access is denied at the NATS layer before reaching the application:

Listing 9: Example agent NATS ACL (Vault-issued)

```

agent: sha256-a3f1c9-node-prod-07
publish:
  allow:
    - beacon.agent.sha256-a3f1c9-node-prod-07.heartbeat
    - beacon.agent.sha256-a3f1c9-node-prod-07.metrics
    - beacon.agent.sha256-a3f1c9-node-prod-07.logs
    - beacon.agent.sha256-a3f1c9-node-prod-07.audit
    - beacon.agent.sha256-a3f1c9-node-prod-07.execution.result
subscribe:
  allow:
    - beacon.agent.sha256-a3f1c9-node-prod-07.commands
    - beacon.agent.sha256-a3f1c9-node-prod-07.config.update
    - beacon.group.production.commands
    - beacon.all.commands

```

14 Queue Manager Engine

14.1 Outbound Command Queue

All commands dispatched to agents pass through the server-side queue manager before entering NATS. This ensures ordering, deduplication, and delivery tracking:

State	Description
Pending	Command assembled; not yet sent to NATS
Dispatched	Published to NATS; awaiting agent ACK
Acknowledged	Agent confirmed receipt
Failed	NATS delivery failed or agent NACK
DeadLetter	Exhausted <code>max_retries</code> ; requires manual inspection

```

atlas queue status
atlas queue pause
atlas queue resume
atlas queue retry-failed
atlas queue clear --state dead-letter

```


15 Encryption Engine

Layer	Algorithm / Mechanism
Transport (server → agents)	TLS 1.3 over NATS; certificate pinning
Transport (operator → server)	TLS 1.3 via Nginx; HSTS enforced
Payload encryption (optional)	AES-256-GCM on NATS message bodies
Key derivation	Argon2id from master password
Secrets storage	HashiCorp Vault (Transit & KV engines)
Agent certificates	Vault PKI; short-lived; auto-rotated
Audit log at rest	AES-256-GCM column-level encryption

15.1 Key Rotation

Key rotation on the server is an atomic operation with rollback support at every step:

1. Generate new AES-256-GCM key via Vault Transit
2. Decrypt all affected records with the current key
3. Re-encrypt with the new key
4. Verify integrity of all re-encrypted records
5. Commit rotation; retire old key version
6. Record rotation event in immutable audit log

16 REST API Engine

All endpoints are RBAC-protected server-side. Everything accessible via the TUI or CLI is available remotely through the REST API with identical access controls.

Method	Endpoint	Min Role	Description
POST/GET	/api/v1/auth	—	Login, logout, token refresh
GET	/api/v1/users	Captain	List all users
POST	/api/v1/users	Captain	Create user
PUT	/api/v1/users/:id	Captain	Edit user
DELETE	/api/v1/users/:id	Captain	Delete user
GET	/api/v1/agents	Viewer	List agents and health
GET	/api/v1/agents/:id	Viewer	Agent detail
PUT	/api/v1/agents/:id	Captain	Edit agent (tags, state)
DELETE	/api/v1/agents/:id	Captain	Deregister agent
GET	/api/v1/namespaces	Viewer	List namespaces
POST	/api/v1/namespaces	Captain	Create namespace
PUT	/api/v1/namespaces/:id	Captain	Publish new version
DELETE	/api/v1/namespaces/:id	Captain	Archive namespace
GET	/api/v1/groups	Viewer	List groups
POST	/api/v1/groups	Captain	Create group
PUT	/api/v1/groups/:id	Captain	Edit group / reorder
GET	/api/v1/executions	Viewer	List executions
POST	/api/v1/executions	Commander	Launch execution
PUT	/api/v1/executions/:id	Commander	Pause / resume / stop
GET	/api/v1/logs	Viewer	Query aggregated logs
GET	/api/v1/metrics	Viewer	Query fleet metrics
GET	/api/v1/audit	Captain	Query immutable audit log
GET	/api/v1/config	Captain	Server configuration
PUT	/api/v1/config	Captain	Update configuration
POST	/api/v1/backups	Captain	Trigger backup
GET	/api/v1/backups	Captain	List backups
POST	/api/v1/backups/:id/restore	Captain	Restore backup

17 WebSocket Gateway Engine

The WebSocket Gateway provides live streaming to operators and external tooling. Clients subscribe to specific channels with optional filters rather than receiving a full broadcast.

Listing 10: WebSocket channel subscription

```
{
  "action":      "subscribe",
  "channel":     "logs",
  "filters": {
    "agent_id":  "sha256:a3f1c9...",
    "namespace": "update_system",
  }
}
```

```

    "severity":    ["error", "critical"]
  }
}

```

17.1 Available Channels

Channel	Content
logs	Structured log entries from all agents (filterable)
metrics	Real-time metric streams per agent or fleet
executions	Execution state transitions and progress events
audit	Audit events (Captain role required)
health	Agent health state changes and heartbeat status
alerts	Threshold breach and anomaly alerts

17.2 Back-pressure and Flood Protection

- Per-client send buffer with configurable cap
- Slow consumers receive frame-drop notifications; oldest frames dropped
- Rate limiting per connection enforced at the Nginx layer
- Reconnect storms suppressed via server-side exponential backoff hint
- Maximum concurrent WebSocket clients: configurable (`max_ws_clients`)

18 Health Monitor Engine

The Health Monitor Engine tracks both the state of the ATLAS Server itself and the aggregate health of the managed agent fleet.

18.1 Server Health States

State	Description
BOOTING	Server startup sequence in progress
INITIALIZING	First-time setup or database migration
ONLINE	All engines operational; all agents reachable
DEGRADED	One or more engines impaired; partial fleet connectivity
RECOVERING	Restoring from failure or backup
SHUTTING_DOWN	Graceful shutdown; draining connections

18.2 Agent Fleet Health Summary

The server exposes a fleet-level health summary via REST and the TUI dashboard:

Listing 11: Fleet health summary endpoint response

```

{
  "total_agents":    147,
  "online":          142,
  "degraded":        3,
  "offline_buffering": 1,
  "unreachable":     1,

```

```

"executions_running": 12,
"executions_queued": 4,
"queue_dead_letter": 0,
"server_state": "ONLINE"
}

```

19 TUI Design

The ATLAS Server TUI is built with **Ratatui** and provides full fleet management capability equivalent to the REST API. All views are role-aware: menu items and actions are hidden or disabled based on the authenticated operator's role.

19.1 View Inventory

- Fleet Dashboard
- Agent List
- Agent Detail
- Namespaces
- Groups
- Executions
- Fleet Metrics
- Log Stream
- Audit Log
- Queue State
- Scheduler
- Users
- Configuration
- Server Health
- Backup Manager

19.2 TUI Requirements

- Full keyboard navigation; mouse optional
- Live data updates backed by WebSocket subscriptions
- Search and filtering on all list views
- Role-aware rendering: Viewers see no execution controls
- Contextual help panel on every view (? key)
- Inline execution launch with target selection (agent / tag / all)

20 CLI Reference

20.1 Installation & Service

```

atlas install / uninstall
atlas service install / remove / start / stop / restart / status

```

20.2 Initialization

```

atlas init
# Prompts:
#   Master Password      Captain Password
#   Commander Password  Viewer Password
#   Domain / Server IP   HTTP Port / HTTPS Port / WebSocket Port
#   NATS address         PostgreSQL DSN
#   Metrics interval    Log sync interval
#   Polling interval     Backup schedule

```

20.3 Authentication

```

atlas login / logout / whoami
atlas password change captain / commander / viewer

```

```
atlas password reset captain / commander / viewer
```

20.4 Agent Management

```
atlas agent list
atlas agent show <agent-id>
atlas agent enable <agent-id>
atlas agent disable <agent-id>
atlas agent remove <agent-id>
atlas agent rename <agent-id> <new-name>
atlas agent tag <agent-id> add <tag>
atlas agent tag <agent-id> remove <tag>
atlas agent regenerate-id <agent-id>
```

20.5 Namespace Management

```
atlas namespace create
atlas namespace edit <name>
atlas namespace delete <name>
atlas namespace list
atlas namespace show <name> [--version v2]
atlas namespace export <name> --output ns.json
atlas namespace import --file ns.json
atlas namespace run <name> [--agent <id>] [--tag <tag>]
atlas namespace stop <execution-id>
atlas namespace pause <execution-id>
atlas namespace resume <execution-id>
```

20.6 Group Management

```
atlas group create
atlas group edit <name>
atlas group delete <name>
atlas group reorder <name>
atlas group add-namespace <group> <namespace>
atlas group remove-namespace <group> <namespace>
atlas group run <name> [--tag <tag>]
```

20.7 User Management

```
atlas user create
atlas user edit <username>
atlas user delete <username>
atlas user enable <username>
atlas user disable <username>
atlas user list
atlas user role assign <username> <role>
```

20.8 Scheduler

```
atlas schedule create
atlas schedule edit <schedule-id>
```

```
atlas schedule delete <schedule-id>
atlas schedule enable <schedule-id>
atlas schedule disable <schedule-id>
atlas schedule list
atlas schedule show <schedule-id>
```

20.9 Database Operations

```
atlas db status / compact / vacuum / verify
atlas db backup / restore <backup-id>
atlas db export --format sql --output dump.sql
atlas db clear / reset
```

20.10 Queue, Encryption, API, WebSocket

```
atlas queue status / pause / resume / retry-failed / clear

atlas encryption enable / disable / rotate-key / status

atlas api start / stop / restart / status

atlas websocket enable / disable / status
```

20.11 Connectivity & TUI

```
atlas server ping / test / status
atlas nats status / test / reconnect
atlas tui
```

21 Security Architecture

Security is structural, not a layer on top. Every inter-component communication is authenticated. No command reaches an agent without passing RBAC validation, receiving a Vault-signed payload, and being recorded in the immutable audit log.

21.1 Defense-in-Depth Layers

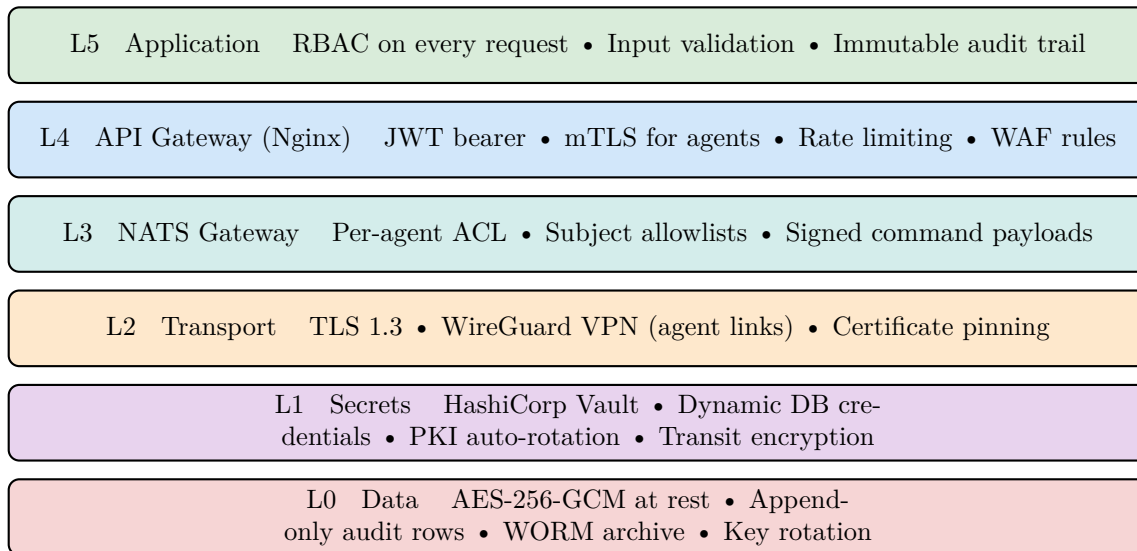


Figure 6: Defense-in-Depth Security Architecture

21.2 Brute Force and Session Protections

- Progressive backoff with account lockout after configurable failed attempts
- Password reset rate-limited; all attempts recorded in audit log
- JWT JTI claim tracking prevents session replay
- Recovery keys invalidated on first use; new key issued
- All administrative operations require re-authentication after session idle

22 Edge Cases & Failure Handling

22.1 Agent Connectivity

- **Agent goes offline mid-execution** — Server marks execution **Interrupted**; agent's local Recovery Engine handles resume on reconnect; server reconciles state via JetStream replay.
- **Agent reconnects after long outage** — JetStream durable consumer replays all missed commands in order; duplicate-delivery deduplication applied by agent.
- **NATS partitioned from server** — Server marks affected agents **UNREACHABLE**; queues commands in PostgreSQL pending queue until NATS recovers.
- **Agent ID collision on re-registration** — Server rejects duplicate **agent_id**; Captain must explicitly deregister old record before new registration is accepted.

22.2 Execution Coordination

- **Fan-out to 1000 agents simultaneously** — Execution Coordinator batches NATS publishes; back-pressure from slow consumers does not block other agents.
- **Partial fleet failure during group run** — Per-agent outcomes recorded independently; group execution marked **PartialSuccess**; rollback policy applied per agent.
- **Server restart during active execution** — Execution records are persisted to PostgreSQL before dispatch; server re-evaluates in-flight executions on startup via JetStream

replay.

- **Namespace version published during active run** — Running executions are pinned to their version; new version visible for subsequent runs only.

22.3 Database and Storage

- **PostgreSQL unavailable** — Server enters `DEGRADED`; read-only cached state served from Redis; write operations queued locally.
- **Disk full on server** — Log and metrics ingestion degrades gracefully; critical path (RBAC, execution dispatch) continues; alert emitted immediately.
- **Backup restore schema mismatch** — Schema version validation rejects incompatible restores; migration path required for version upgrades.

22.4 WebSocket and API

- **WebSocket reconnect storm** — Server-side exponential backoff hint sent in disconnect message; Redis rate-limiter enforces reconnect ceiling.
- **API flooded with execution requests** — Global resource limit (`max_parallel_executions`) enforced; excess requests return `429 Too Many Requests`.
- **Clock skew between server and agents** — All timestamps are UTC; monotonic clocks used for duration measurements; NTP sync required for fleet agents.

23 Middleware Stack

Component	Technology	Role
Reverse Proxy	Nginx 1.26	TLS 1.3 termination, request routing, WAF, rate limiting
Cache / Rate-limit	Redis 7	JWT session cache, rate-limit token buckets, pub/sub signalling
Message Bus	NATS JetStream 2.10	All agent-server communication; durable streams; ACL enforcement
Secrets Manager	HashiCorp Vault 1.17	Dynamic secrets, PKI for agent mTLS, JWT signing, DB credentials
Metrics	Prometheus + Grafana	Fleet time-series, recording rules, dashboards, alert routing
Relational DB	PostgreSQL 16	All server-side persistent state
Connection Pool	PgBouncer	PostgreSQL connection multiplexing
Container Runtime	Docker / Podman	Isolated service containers; cgroups v2 resource limits

Table 2: ATLAS Server Middleware Stack

24 Deployment Resource Budget

Estimates for a fleet of 100 agents. Single 16 GB control-plane host is sufficient for this scale.

Component	CPU (cores)	Memory	Notes
Nginx	0.5	128 MB	Scales with concurrent connections
Redis 7	0.5	256 MB	Session + rate-limit store
NATS JetStream	1.0	512 MB	Scales with stream count and message rate
HashiCorp Vault	0.5	256 MB	PKI + Transit + KV
Prometheus	1.0	1 GB	100 agents, 15s scrape, 365d retention
Grafana	0.5	512 MB	Dashboard and alert UI
PostgreSQL 16	2.0	2 GB	Fleet state + audit; WAL archiving
PgBouncer	0.2	64 MB	Connection pooling
ATLAS Server binary	2.0	1 GB	All engines; async Tokio runtime
Total	~8.2 cores	~5.7 GB	Comfortable on 16 GB host

Table 3: Central Control Plane Resource Budget (100-agent fleet)

25 Non-Functional Requirements

Requirement	Specification
Language	Rust only; no C FFI in core paths
Runtime	Tokio async runtime; event-driven throughout
Scalability	1 to 1000+ agents from a single server instance without architectural changes
Availability	Graceful shutdown; in-flight executions drained before stop
Crash Recovery	All state in PostgreSQL; server restarts reconcile from JetStream
Security	Secure by default; no insecure configuration available without explicit override
Testing	Unit, integration, property-based, and chaos tests
Observability	Prometheus metrics and structured logs for every engine
Auditability	Every action traceable; immutable audit trail; no exceptions
Parity	Every CLI command available via REST API; every API endpoint in TUI
Multi-server	Single-server initial release; federation support in roadmap
RBAC	Extensible to custom roles in future releases
Plugin System	Exporter and notification plugins via first-class plugin registry

26 Implementation Roadmap

Phase	Deliverables
0 Infrastructure	Docker Compose stack: Nginx, Redis, NATS JetStream, Vault, PostgreSQL, Prometheus, Grafana; CI/CD pipeline; schema migrations
1 Auth & RBAC	Authentication Engine, Authorization Engine, JWT/refresh token flow, Argon2id password store, recovery key generation
2 Agent Registry	Agent Registry Engine, NATS Gateway Engine with ACL enforcement, agent registration, heartbeat monitoring
3 Namespaces	Namespace Manager Engine, versioning, group composition, namespace CRUD, import/export
4 Execution	Execution Coordinator Engine, fan-out dispatch, lock manager, state machine, per-agent result tracking
5 Scheduler	Scheduler Engine, cron/interval support, conflict and missed-run policies, maintenance windows
6 Telemetry	Metrics Aggregation Engine, Log Aggregation Engine, fleet rollups, Prometheus integration
7 Audit & Encryption	Immutable Audit Engine, Encryption Engine, Vault Transit integration, key rotation
8 REST API & WS	REST API Engine, WebSocket Gateway, full endpoint coverage, rate limiting
9 TUI	Ratatui TUI, all views, role-aware rendering, live WebSocket updates
10 Hardening	Penetration test, load test at 1000 agents, RBAC review, WORM archive, runbook documentation

Table 4: Phased Implementation Roadmap

27 Conclusion

ATLAS Server is the central authority of the Beacon fleet management system. It delivers:

- **Unified fleet control** — one server manages 1 to 1 000+ Beacon agents through NATS JetStream without architectural changes.
- **Strict security boundaries** — RBAC validated server-side at every request; agents never make authorisation decisions; every command is signed by Vault.
- **Immutable audit trail** — append-only PostgreSQL table with database-level protections; no operator role can modify records.
- **Offline-resilient architecture** — JetStream durable streams deliver queued commands to agents on reconnect; server state survives restarts via PostgreSQL.
- **Approved operations only** — no arbitrary shell execution at any layer; all commands flow through versioned namespace definitions.
- **Production-grade observability** — Prometheus metrics, Grafana dashboards, structured log aggregation, and WebSocket live streams for every engine.
- **Realistic resource footprint** — approximately 5.7 GB RAM for a 100-agent fleet; deployable on a single 16 GB host.

Together, ATLAS Server and the Beacon agent form a cohesive, auditable, offline-capable fleet

operations platform comparable in architecture to components found in Salt, Ansible, Rundeck, Zabbix, and Prometheus — while remaining entirely Rust-native, purpose-built, and operationally lean.